LLM Gateway — Authentication & Security

Interview preparation · V1 · gateway vs provider keys, SHA-256, Bearer auth, secret isolation

1. Two Kinds of API Keys

The single most important security distinction in the project.

Q1. Explain the difference between the gateway API key and the provider API keys.

EASY

Answer: The gateway API key authenticates a CLIENT to OUR gateway and is sent as Authorization: Bearer gw_... . The provider API keys (Gemini, Groq) authenticate the GATEWAY to the external providers and live only in the backend. They are completely separate credentials and must never be mixed.

Simple explanation: Client key = 'who may call us'. Provider key = 'how we call the LLM'.

Project example: The React frontend only ever handles the gateway key (typed by the user). Provider keys stay in backend/.env and never reach the browser bundle.

```
Client  --(Bearer gw_...)-->  Gateway  --(provider key)-->  Gemini/Groq
```

Interview tip: This is almost always asked. Say it crisply and give the 'never in the frontend' line.

Q2. How do you make sure provider keys never leak to the frontend?

MEDIUM

Answer: Provider keys are read only by backend config and used only inside adapters, sent in request headers (never logged, never in URLs). The frontend bundle is built with only VITE_API_BASE_URL (a non-secret). We scanned the built bundle and image to confirm no provider keys or the gateway key appear.

Simple explanation: Keys stay server-side; the browser gets code, not secrets.

Project example: VITE_* values are embedded in the browser bundle, so by rule only the non-secret base URL goes there; everything sensitive is backend-only.

Likely follow-ups:

- Why can't VITE_* hold secrets? (anything in the frontend build ships to every browser)

2. How Authentication Works

Validating a client key on every protected request.

Q3. Walk through what happens when a request with a gateway key arrives.

MEDIUM

Answer: A FastAPI dependency, require_api_key, extracts the Bearer token using HTTPBearer(auto_error=False). It hashes the token with SHA-256 and looks up an active api_keys row by that hash. If found, it resolves the owning user into a ClientIdentity; if missing/inactive, it raises AuthenticationError -> 401.

Simple explanation: Hash the presented key, look it up, allow if it matches an active key.

```
require_api_key
  -> HTTPBearer parses 'Authorization: Bearer ...'
  -> sha256(token) -> lookup active api_keys row
  -> found: ClientIdentity   |   not found: 401 UNAUTHORIZED
```

Interview tip: Mention auto_error=False: it lets you return your own standardized 401, not FastAPI's default 403.

Q4. Why store only a hash of the key, not the key itself?

MEDIUM

Answer: If the database ever leaks, stored hashes are useless to an attacker, whereas raw keys would be instant account takeover. We store sha256(key) and compare hashes on each request; the raw key is never persisted anywhere.

Simple explanation: A stolen hash can't be used to call the API; a stolen raw key can.

Project example: The seed command reads GATEWAY_API_KEYS, hashes each, and stores only the 64-char hex digest. Verified in the DB that no gw_ prefix is stored.

Likely follow-ups:

- Why SHA-256 and not bcrypt/argon2? (next question)

Q5. Why SHA-256 for API keys instead of bcrypt/argon2 like passwords?

HARD

Answer: Passwords are short and low-entropy, so they need deliberately slow, salted hashing to resist brute force. API keys here are long, random, high-entropy secrets, so a fast one-way hash is sufficient — and fast matters because we hash on every request. Slow hashing would add latency with no real security gain for high-entropy tokens.

Simple explanation: Random long keys don't need slow hashing; passwords do.

Project example: gw_ keys are generated with secrets.token_hex, so guessing is infeasible; SHA-256 keeps per-request auth cheap.

Interview tip: This trade-off question is a favorite — the entropy argument is the key insight.

Q6. What is a Bearer token and why 401 (not 403) for a bad key?

EASY

Answer: A Bearer token is a credential sent as-is in the Authorization header — 'whoever bears it gets access'. A missing or invalid key means we don't know who you are, which is 401 Unauthorized. 403 Forbidden would mean 'we know you but you may not do this' — that's authorization/permissions, which is out of V1 scope.

Simple explanation: 401 = 'who are you?'. 403 = 'you can't do that'. Bad key is a 401.

Project example: AuthenticationError maps to 401 UNAUTHORIZED through the centralized error handler.

---

3. Secrets & Design Boundaries

Configuration, isolation, and what auth deliberately does NOT do.

Q7. How is authentication kept separate from business logic?

MEDIUM

Answer: All auth lives in one module and is attached as a route dependency. GatewayService and the provider adapters never import or know about auth; they only receive data. The identity needed for logging is passed in from the route, not pulled into the gateway.

Simple explanation: Auth is a gate in front of the route; the core never sees it.

Project example: A grep of services/ and providers/ shows no auth imports; the gateway stays auth-agnostic.

Likely follow-ups:

- Why does that matter? (the gateway can be tested with zero auth/DB setup)

Q8. What are the security limits of V1, honestly?

MEDIUM

Answer: There is no RBAC/permissions, no rate limiting, no key rotation workflow beyond re-seeding, and no per-key quotas. Auth is intentionally authentication-only (identity), not authorization. These are deliberate V2 concerns, not bugs.

Simple explanation: V1 answers 'who are you', not 'what are you allowed to do or how much'.

Project example: Revoking a key is supported today by setting is_active=false, which makes lookups treat it as unknown (401).

Interview tip: Stating limits plainly builds trust; then mention is_active revocation as the one control you do have.